

UNIVERSIDAD SERGIO ARBOLEDA

PARCIAL 2: ALGORITMOS EVOLUTIVOS, GENÉTICOS, ASCENSO DE COLINAS, TEMPLE SIMULADO, ACO Y PSO

Fundamentos de Inteligencia Artificial

Inteligencia Artificial (SIST5036)

Camilo Andrés Díaz García

Juan David Andradé Gómez

Mario Jiménez López

Profesor: Joaquín F. Sánchez

Universidad Sergio Arboleda
Facultad de Sistemas e Ingeniería
Bogotá, octubre de 2026

Índice

1. Punto 1. Algoritmos evolutivos y algoritmos genéticos **2**
1.1. Parte A: componente matemático y conceptual 2
1.2. Parte B: implementación en Python 3
1.2.1. Decisiones de implementación 3
1.2.2. Código 3
1.2.3. Resultados 5
1.2.4. Pregunta de interpretación 7

2. Punto 2. Ascenso de colinas y óptimos locales **7**
2.1. Parte A: componente matemático y conceptual 8
2.2. Parte B: implementación en Python 9
2.2.1. Decisiones de implementación 9
2.2.2. Código 9
2.2.3. Resultados 10
2.2.4. Pregunta de interpretación 12

3. Punto 3. Temple simulado: aceptación de soluciones peores **12**
3.1. Parte A: componente matemático y conceptual 12
3.2. Parte B: implementación en Python 13
3.2.1. Decisiones de implementación 13
3.2.2. Código 13
3.2.3. Resultados 15
3.2.4. Pregunta de interpretación 17

4. Punto 4. ACO: elección de rutas y actualización de feromonas **18**
4.1. Parte A: componente matemático y conceptual 18
4.2. Parte B: implementación en Python 19
4.2.1. Decisiones de implementación 19
4.2.2. Código 19
4.2.3. Resultados 21
4.2.4. Pregunta de interpretación 23

5. Punto 5. PSO: movimiento de partículas y memoria **24**
5.1. Parte A: componente matemático y conceptual 24
5.2. Parte B: implementación en Python 25
5.2.1. Decisiones de implementación 25
5.2.2. Código 25
5.2.3. Resultados 27
5.2.4. Pregunta de interpretación 28

6. Conclusiones **29**

1 Punto 1. Algoritmos evolutivos y algoritmos genéticos

Se maximiza $f(x) = x^2$ con $x \in \{0, \ldots, 31\}$. Cada solución es un cromosoma de cinco bits. La población inicial es $P = \{\texttt{00101}, \texttt{01100}, \texttt{10010}, \texttt{11001}\}$.

1.1 Parte A: componente matemático y conceptual

1. Decodificación y aptitud. Cada cromosoma se lee como un número binario. Por ejemplo, $\texttt{00101} = 0 \cdot 16 + 0 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 5$ y su aptitud es $5^2 = 25$. Los demás se calculan igual (Tabla 1).
2. Probabilidad de ruleta. Cada individuo recibe una fracción de la ruleta proporcional a su aptitud. La suma de las aptitudes es 1118. El individuo `11001` tiene la mayor probabilidad (0,5590) porque su aptitud es la más alta.

Cromosoma	x	$f(x)$	**Probabilidad de ruleta**
`00101`	5	25	0,0224
`01100`	12	144	0,1288
`10010`	18	324	0,2898
`11001`	25	625	0,5590
Suma		1118	1,0000

Tabla 1: Decodificación, aptitud y probabilidad de selección por ruleta de la población inicial.

3. Cruce. Los padres son `01100` y `11001`. Se cortan después del segundo bit: `01|100` y `11|001`. Cada descendiente toma el inicio de un padre y el final del otro: `01001` y `11100`.
4. Mutación. Al invertir el último bit de `01001` se obtiene `01000`. Su valor es $x = 8$ y su aptitud baja de 81 a 64 (Tabla 2).

Individuo	**Cromosoma**	x	$f(x)$
Padre 1	`01100`	12	144
Padre 2	`11001`	25	625
Descendiente 1	`01001`	9	81
Descendiente 2	`11100`	28	784
Descendiente 1 mutado	`01000`	8	64

Tabla 2: Cruce de un punto tras el segundo bit y mutación del último bit del primer descendiente.

5. Explicación breve.

- **Evolutivo y genético.** Un algoritmo evolutivo mejora una población de soluciones por selección y variación a lo largo de generaciones. El algoritmo genético es una familia dentro de los evolutivos: representa cada solución como un cromosoma y usa selección, cruce y mutación.
- **Selección, cruce y mutación.** La selección decide quiénes se reproducen y favorece a los de mayor aptitud. El cruce combina material de dos padres para probar soluciones nuevas. La mutación cambia bits al azar y evita que la población se quede sin variedad.

- **Mutación útil aunque empeore.** Un cambio que baja la aptitud inmediata puede introducir un valor que ya no existe en la población. Un dato de este problema lo muestra: el cuarto bit vale 1 solo en `10010`. Si ese cromosoma desaparece, ningún cruce puede devolver ese bit, porque el cruce solo mezcla bits que ya existen. El óptimo `11111` no se obtiene con un solo cruce entre los cuatro cromosomas iniciales (comprobado por código). Con cruces repetidos sí es alcanzable, pero solo mientras `10010` siga en la población. Cuando se pierde, solo la mutación puede recuperar ese bit. La ejecución de la parte B lo confirma (Sección 1.2.4).

1.2 Parte B: implementación en Python

1.2.1 Decisiones de implementación

Aspecto	Supuesto adoptado
Generador	`random.Random(42)`, creado una sola vez al inicio de `algoritmo_genetico`.
Nueva generación	El mejor individuo pasa sin cambios (elitismo; en un empate se conserva el primero). Los otros tres se generan por ruleta, cruce y mutación.
Ruleta	Con reemplazo: los dos padres pueden ser el mismo individuo. Usa un número `random()` multiplicado por la suma de aptitudes y recorre el acumulado.
Cruce	Una llamada `random()` decide si ocurre ($p < 0{,}8$). Si ocurre, `randint(1, 4)` fija el punto. Si no, los hijos son copias de los padres.
Mutación	Cinco llamadas `random()` por hijo, una por bit, de izquierda a derecha. Se invierte el bit si el número es menor que 0,05.
Descendientes	Se producen en pares. Como faltan tres, el segundo par aporta un solo hijo y el sobrante se descarta antes de mutarlo, sin consumir llamadas.
Orden por par	Padre 1, padre 2, decisión de cruce, punto de cruce, mutación del hijo 1 y mutación del hijo 2.
Historial	Mejor aptitud en la generación 0 (población inicial) y tras cada una de las 20 generaciones: 21 valores.

Tabla 3: Decisiones de implementación del punto 1.

1.2.2 Código

```
"""Punto 1: algoritmo genetico para maximizar f(x) = x^2, x en 0..31."""
import json
import random
from pathlib import Path

POBLACION_INICIAL = ["00101", "01100", "10010", "11001"]
PROB_CRUCE = 0.8
PROB_MUTACION = 0.05
GENERACIONES = 20
SEMILLA = 42
OPTIMO = 31 ** 2
CARPETA_SALIDA = Path(__file__).resolve().parent.parent / "outputs"

```

```
14
15 def decodificar(cromosoma):
16     """Valor entero de un cromosoma binario."""
17     return int(cromosoma, 2)
18
19
20 def aptitud(cromosoma):
21     """Aptitud f(x) = x^2."""
22     return decodificar(cromosoma) ** 2
23
24
25 def ruleta(poblacion, generador):
26     """Selecciona un individuo con probabilidad proporcional a su aptitud."""
27     aptitudes = [aptitud(c) for c in poblacion]
28     umbral = generador.random() * sum(aptitudes)
29     acumulado = 0
30     for cromosoma, valor in zip(poblacion, aptitudes):
31         acumulado += valor
32         if umbral < acumulado:
33             return cromosoma
34     return poblacion[-1]
35
36
37 def cruce_un_punto(padre1, padre2, generador):
38     """Cruce de un punto con probabilidad PROB_CRUCE."""
39     if generador.random() < PROB_CRUCE:
40         punto = generador.randint(1, len(padre1) - 1)
41         return (padre1[:punto] + padre2[punto:],
42                 padre2[:punto] + padre1[punto:])
43     return padre1, padre2
44
45
46 def mutar(cromosoma, generador):
47     """Invierte cada bit de forma independiente con PROB_MUTACION."""
48     bits = []
49     for bit in cromosoma:
50         if generador.random() < PROB_MUTACION:
51             bit = "1" if bit == "0" else "0"
52         bits.append(bit)
53     return "".join(bits)
54
55
56 def nueva_generacion(poblacion, generador):
57     """Elitismo mas tres descendientes por ruleta, cruce y mutacion."""
58     siguiente = [max(poblacion, key=aptitud)]
59     while len(siguiente) < len(poblacion):
60         padre1 = ruleta(poblacion, generador)
61         padre2 = ruleta(poblacion, generador)
62         hijo1, hijo2 = cruce_un_punto(padre1, padre2, generador)
63         siguiente.append(mutar(hijo1, generador))
64         if len(siguiente) < len(poblacion):
65             siguiente.append(mutar(hijo2, generador))
66     return siguiente
67
68
69 def algoritmo_genetico():
70     """Ejecuta las 20 generaciones y devuelve el registro completo."""
71     generador = random.Random(SEMILLA)
72     poblacion = list(POBLACION_INICIAL)
73     poblaciones = [list(poblacion)]
74     historial = [max(aptitud(c) for c in poblacion)]
75     for _ in range(GENERACIONES):
76         poblacion = nueva_generacion(poblacion, generador)
77         poblaciones.append(list(poblacion))
78         historial.append(max(aptitud(c) for c in poblacion))
79     mejor = max(poblacion, key=aptitud)
80     return {
81         "semilla": SEMILLA,
82         "poblaciones": poblaciones,
83         "historial_mejor_aptitud": historial,
84         "distintos_por_generacion": [len(set(p)) for p in poblaciones],
85         "poblacion_final": poblacion,
86         "aptitudes_finales": [aptitud(c) for c in poblacion],
```

```
        "mejor_cromosoma": mejor,
        "mejor_x": decodificar(mejor),
        "mejor_aptitud": aptitud(mejor),
        "optimo_global": OPTIMO,
        "encontro_optimo": aptitud(mejor) == OPTIMO,
    }


def main():
    resultados = algoritmo_genetico()
    CARPETA_SALIDA.mkdir(exist_ok=True)
    ruta = CARPETA_SALIDA / "punto1_resultados.json"
    ruta.write_text(json.dumps(resultados, indent=2), encoding="utf-8")
    print("Mejor cromosoma final:", resultados["mejor_cromosoma"])
    print("x =", resultados["mejor_x"])
    print("Aptitud =", resultados["mejor_aptitud"])
    print("Historial:", resultados["historial_mejor_aptitud"])
    print("Poblacion final:", resultados["poblacion_final"])
    print("Encontro el optimo:", resultados["encontro_optimo"])


if __name__ == "__main__":
    main()
```

Código 1: Implementación del punto 1 (`punto1_genetico.py`).

1.2.3 Resultados

Mejor cromosoma	x	Aptitud	Máximo posible	Óptimo hallado
11111	31	961	961	Sí

Tabla 4: Mejor individuo de la población final (generación 20).

- **Estructura:** una fila con el mejor individuo de la población final y el máximo teórico.
- **Qué se ve:** el cromosoma 11111, $x = 31$ y aptitud 961, igual al máximo posible 961.
- **Por qué:** permite comparar de inmediato lo hallado con el óptimo global.

Gen.	Mejor aptitud	Gen.	Mejor aptitud	Gen.	Mejor aptitud
0	625	7	841	14	841
1	841	8	841	15	841
2	841	9	841	16	961
3	841	10	841	17	961
4	841	11	841	18	961
5	841	12	841	19	961
6	841	13	841	20	961

Tabla 5: Historial de la mejor aptitud por generación (la generación 0 es la población inicial).

- **Estructura:** tres bloques de columnas (generación y mejor aptitud) leídos de arriba hacia abajo y de izquierda a derecha.
- **Qué se ve:** la aptitud parte en 625, sube a 841 en la generación 1, se mantiene hasta la generación 15 y llega a 961 en la generación 16. Nunca baja.
- **Por qué:** el elitismo conserva siempre al mejor, así que la mejor aptitud no puede decrecer.

Gen.	Población	Distintos	Gen.	Población	Distintos
0	00101 01100 10010 11001	4	11	11101 11101 11001 11101	2
1	11001 11100 11001 11101	3	12	11101 11101 11101 11101	1
2	11101 11100 11001 11101	3	13	11101 11101 11101 11101	1
3	11101 11101 11101 11101	1	14	11101 11101 11101 11101	1
4	11101 11101 11101 11101	1	15	11101 10101 11101 11101	2
5	11101 11101 01101 01101	2	16	11101 11101 11111 01101	3
6	11101 11101 01001 11101	2	17	11111 00101 11101 01111	4
7	11101 11101 11101 11101	1	18	11111 11111 01111 11111	2
8	11101 11101 11101 11101	1	19	11111 01101 01111 11111	3
9	11101 11101 11101 11101	1	20	11111 11111 11111 11111	1
10	11101 01101 11101 11101	2			

Tabla 6: Población de cada generación y número de cromosomas distintos.

- **Estructura:** dos bloques de columnas con la generación, los cuatro cromosomas y cuántos son distintos.
- **Qué se ve:** entre las generaciones 3 y 15 hay como máximo 2 cromosomas distintos. Entre las generaciones 1 y 15 ningún individuo tiene el cuarto bit en 1. En la generación 16 aparece `11111`.
- **Por qué:** muestra la pérdida de diversidad y el origen del óptimo, que solo pudo venir de una mutación.

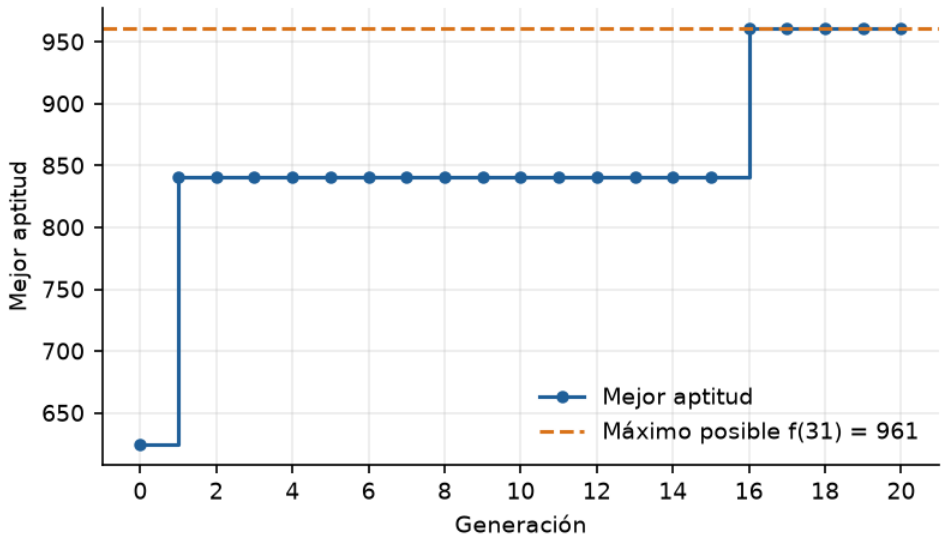


Figura 1: Mejor aptitud por generación frente al máximo posible.

- **Estructura:** eje horizontal con las generaciones, eje vertical con la mejor aptitud y una línea punteada en 961.
- **Qué se ve:** una escalera que sube a 841, se detiene en una meseta y sube al máximo en la generación 16.
- **Por qué:** la meseta indica que el cruce y la selección ya no aportaban mejoras, y el salto final lo produjo una mutación.

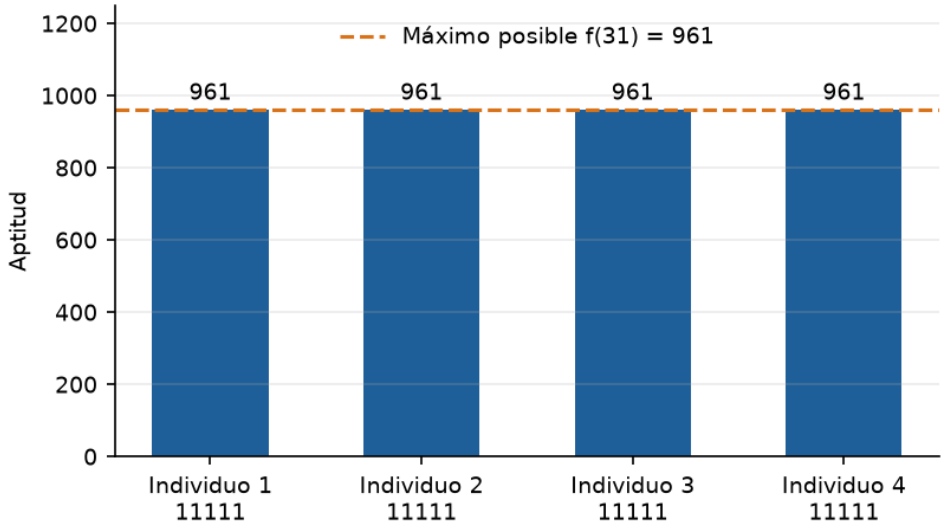


Figura 2: Población final y aptitud de cada individuo.

- **Estructura:** una barra por individuo de la población final, con su cromosoma y su aptitud sobre la barra.
- **Qué se ve:** los cuatro individuos son `11111` con aptitud 961, y todas las barras tocan la línea del máximo.
- **Por qué:** la población final ya no tiene diversidad: los cuatro individuos son idénticos.

1.2.4 Pregunta de interpretación

¿Encontró el óptimo global? Sí. El mejor cromosoma final es `11111`, con $x = 31$ y aptitud 961. El máximo posible de la función es $f(31) = 961$, así que coinciden. Lo encontró en la generación 16, después de 15 generaciones sin mejorar (aptitud 841, que corresponde a $x = 29$).

Por qué veinte generaciones no garantizan hallarlo.

- **Población de solo cuatro individuos.** Con tan pocos individuos la variedad se agota rápido. En 9 de las 20 generaciones los cuatro individuos eran iguales, y cuando los padres son iguales el cruce no cambia nada.
- **Pérdida de diversidad.** `10010` se perdió en la primera generación, y con él el único cuarto bit en 1. Desde ahí, ni la selección ni el cruce podían producir `11111`.
- **Estocasticidad de la mutación.** Solo la mutación podía recuperar ese bit. Si la población fuera toda `11101`, la probabilidad de que un descendiente se volviera `11111` sería $0{,}05 \cdot 0{,}95^4 = 0{,}0407$ (4,1 %), y la de que aparezca al menos uno entre los tres descendientes de una generación, 0,1173 (11,7 %). Con esa simplificación, la probabilidad de no verlo en 20 generaciones sería 0,0825 (8,3 %). Es pequeña, pero no es cero.
- **Alcance del resultado.** Se ejecutó una sola semilla. El éxito observado no demuestra que el algoritmo lo halle siempre; solo muestra que, con esta semilla, la mutación llegó a tiempo.

2 Punto 2. Ascenso de colinas y óptimos locales

Se maximiza $f(x)$ sobre nueve estados, con vecinos $x-1$ y $x+1$ dentro del dominio. El algoritmo elige el vecino de mayor valor y solo se mueve si hay mejora estricta.

2.1 Parte A: componente matemático y conceptual

1. Desde $x_0 = 0$. Los vecinos de 0 son solo el 1, con $f(1) = 4 > f(0) = 1$, así que se mueve al 1. En 1 los vecinos son 0 (1) y 2 (7); gana el 2. En 2 los vecinos son 1 (4) y 3 (5); el mejor es 3, pero $5 < 7$, así que no hay mejora y se detiene. Estados visitados: $0, 1, 2$.
2. Desde $x_0 = 4$. En 4 los vecinos son 3 (5) y 5 (6); gana el 5. En 5 los vecinos son 4 (3) y 6 (9); gana el 6. En 6 los vecinos son 5 (6) y 7 (8); el mejor es $8 < 9$, así que se detiene. Estados visitados: $4, 5, 6$ (Tabla 7).

Inicio x_0	**Estados visitados**	**Estado final**	**f final**
0	$0 \to 1 \to 2$	2	7
4	$4 \to 5 \to 6$	6	9

Tabla 7: Ascenso de colinas calculado desde $x_0 = 0$ y desde $x_0 = 4$.

3. Máximos locales y global. Un máximo local es un estado sin vecinos mejores. Hay dos: $x = 2$ con $f = 7$ y $x = 6$ con $f = 9$. El máximo global es $x = 6$, el de mayor valor de toda la función (Tabla 8).

x	$f(x)$	**Vecinos**	**Clasificación**
0	1	1	Ninguno
1	4	0, 2	Ninguno
2	7	1, 3	Máximo local
3	5	2, 4	Ninguno
4	3	3, 5	Ninguno
5	6	4, 6	Ninguno
6	9	5, 7	Máximo global
7	8	6, 8	Ninguno
8	5	7	Ninguno

Tabla 8: Clasificación de los nueve estados del espacio.

4. Por qué importa el estado inicial. El algoritmo es determinista y voraz: siempre sube por la pendiente más cercana. Cada máximo tiene su propia cuenca de atracción, es decir, el conjunto de inicios que terminan en él. Los inicios 0 a 3 caen en el máximo local 2 y los inicios 4 a 8 en el global 6. Empezar en una u otra cuenca cambia el resultado.
5. Estrategia y costo. La estrategia es el reinicio aleatorio: repetir el ascenso desde varios estados iniciales elegidos al azar y conservar el mejor resultado. El costo es mayor número de evaluaciones de la función, proporcional al número de reinicios. Además, no hay garantía de éxito: todos los reinicios podrían caer en la cuenca del máximo local.

2.2 Parte B: implementación en Python

2.2.1 Decisiones de implementación

Aspecto	Supuesto adoptado
Aleatoriedad	Ninguna. El algoritmo es determinista y no usa generador.
Interfaz	`ascenso_colinas(valores, inicio)` devuelve el estado final y la lista de estados visitados, incluido el inicial.
Vecino elegido	El de mayor valor. Si hubiera un empate, `max` devuelve el primero (el izquierdo); en estos datos no ocurre.
Parada	Se detiene cuando el mejor vecino no supera estrictamente el valor actual.
Reinicios	Se calcula $1-(1-p)^k$ con p igual a la proporción de inicios que llegan al global, para $k = 1,2,3,5$.

Tabla 9: Decisiones de implementación del punto 2.

2.2.2 Código

```
"""Punto 2: ascenso de colinas sobre un espacio discreto."""
import json
from pathlib import Path

VALORES = [1, 4, 7, 5, 3, 6, 9, 8, 5]
REINICIOS = [1, 2, 3, 5]
CARPETA_SALIDA = Path(__file__).resolve().parent.parent / "outputs"


def ascenso_colinas(valores, inicio):
    """Devuelve el estado final y la trayectoria."""
    actual = inicio
    trayectoria = [actual]
    while True:
        vecinos = [v for v in (actual - 1, actual + 1)
                   if 0 <= v < len(valores)]
        mejor_vecino = max(vecinos, key=lambda v: valores[v])
        if valores[mejor_vecino] <= valores[actual]:
            return actual, trayectoria
        actual = mejor_vecino
        trayectoria.append(actual)


def maximos_locales(valores):
    """Estados sin ningun vecino de valor estrictamente mayor."""
    resultado = []
    for x, valor in enumerate(valores):
        vecinos = [v for v in (x - 1, x + 1) if 0 <= v < len(valores)]
        if all(valores[v] <= valor for v in vecinos):
            resultado.append(x)
    return resultado


def ejecutar():
    """Corre el ascenso desde los nueve estados y calcula los reinicios."""
    global_x = max(range(len(VALORES)), key=lambda x: VALORES[x])
    filas = []
    for inicio in range(len(VALORES)):
        final, trayectoria = ascenso_colinas(VALORES, inicio)
        filas.append({
            "inicio": inicio,
            "trayectoria": trayectoria,
            "final": final,
            "valor_final": VALORES[final],
```

```
            "llega_global": final == global_x,
        })
    exitos = sum(fila["llega_global"] for fila in filas)
    p_exito = exitos / len(VALORES)
    return {
        "valores": VALORES,
        "maximos_locales": maximos_locales(VALORES),
        "maximo_global": global_x,
        "filas": filas,
        "exitos": exitos,
        "total": len(VALORES),
        "proporcion": p_exito,
        "reinicios": [
            {"k": k, "prob_al_menos_un_exito": 1 - (1 - p_exito) ** k}
            for k in REINICIOS
        ],
    }


def main():
    resultados = ejecutar()
    CARPETA_SALIDA.mkdir(exist_ok=True)
    ruta = CARPETA_SALIDA / "punto2_resultados.json"
    ruta.write_text(json.dumps(resultados, indent=2), encoding="utf-8")
    for fila in resultados["filas"]:
        print(fila["inicio"], fila["trayectoria"], fila["final"],
              fila["valor_final"])
    print("Proporcion al global:", resultados["exitos"], "de",
          resultados["total"], resultados["proporcion"])
    for fila in resultados["reinicios"]:
        print("k =", fila["k"], fila["prob_al_menos_un_exito"])


if __name__ == "__main__":
    main()
```

Código 2: Implementación del punto 2 (`punto2_ascenso_colinas.py`).

2.2.3 Resultados

Estado inicial	Trayectoria	Estado final	Valor final	¿Global?
0	$0 \to 1 \to 2$	2	7	No
1	$1 \to 2$	2	7	No
2	2	2	7	No
3	$3 \to 2$	2	7	No
4	$4 \to 5 \to 6$	6	9	Sí
5	$5 \to 6$	6	9	Sí
6	6	6	9	Sí
7	$7 \to 6$	6	9	Sí
8	$8 \to 7 \to 6$	6	9	Sí

Tabla 10: Resultado del ascenso de colinas desde los nueve estados iniciales.

- **Estructura:** una fila por estado inicial, con su trayectoria, estado final, valor final y si llegó al global.
- **Qué se ve:** los inicios 0 a 3 terminan en 2 con valor 7; los inicios 4 a 8 terminan en 6 con valor 9.
- **Por qué:** muestra las dos cuencas de atracción obtenidas al ejecutar la función.

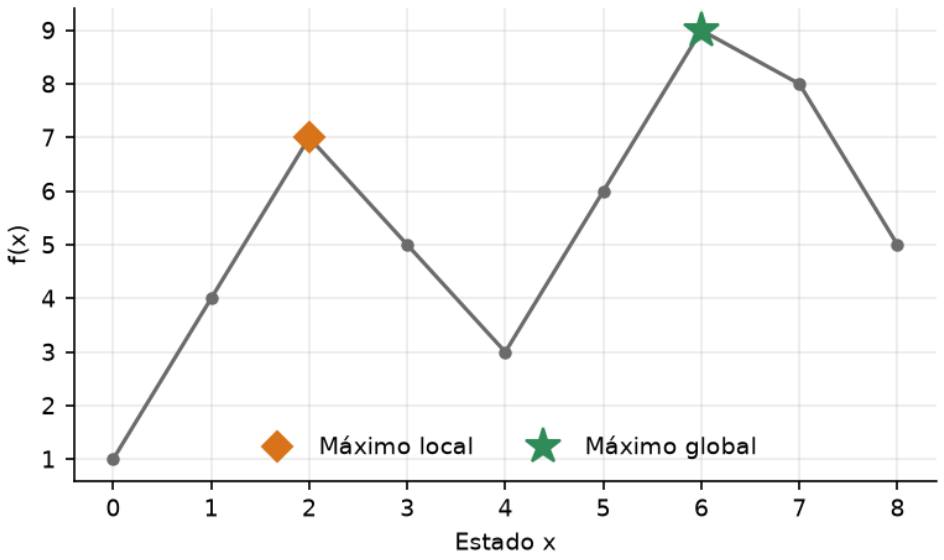


Figura 3: Función $f(x)$ con los máximos local y global marcados.

- **Estructura:** estados en el eje horizontal y valor de la función en el vertical; el rombo naranja es el máximo local y la estrella verde el global.
- **Qué se ve:** dos picos separados por un valle en $x = 4$.
- **Por qué:** el valle impide que un ascenso que empieza a un lado alcance el pico del otro lado.

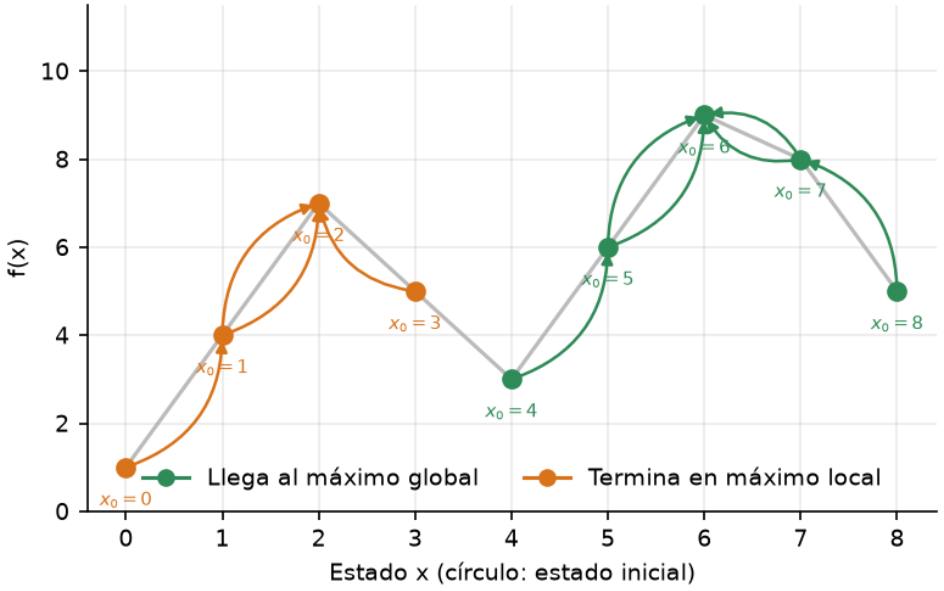


Figura 4: Trayectorias del ascenso de colinas desde los nueve estados iniciales.

- **Estructura:** cada flecha es un movimiento; el círculo marca el estado inicial y el color indica el destino (verde: global, naranja: local).
- **Qué se ve:** los caminos que empiezan entre 0 y 3 suben al pico de $x = 2$; los que empiezan entre 4 y 8 suben al pico de $x = 6$.
- **Por qué:** las flechas se curvan para que se distingan las trayectorias que comparten tramos.

2.2.4 Pregunta de interpretación

Proporción que llega al máximo global. 5 de los 9 estados iniciales (55,6 %) terminan en el global $x = 6$. Los demás terminan en el máximo local $x = 2$.

Relación con los reinicios aleatorios.

- Si el estado inicial se elige de forma uniforme, un solo intento llega al global con probabilidad $5/9$, igual a la proporción observada.
- Con k reinicios independientes, la probabilidad de fallar en todos es $(4/9)^k$, y la de al menos un éxito es $1-(4/9)^k$ (Tabla 11).

Reinicios k	P(**al menos un éxito**)
1	0,5556
2	0,8025
3	0,9122
5	0,9827

Tabla 11: Probabilidad de alcanzar el máximo global con k reinicios independientes desde un estado uniforme.

- **Estructura:** una fila por número de reinicios k con la probabilidad de al menos un éxito.
- **Qué se ve:** pasa de 0,5556 con $k = 1$ a 0,9122 con $k = 3$ y a 0,9827 con $k = 5$.
- **Por qué:** cada reinicio extra reduce por un factor $4/9$ la probabilidad de fallar, pero nunca la lleva a cero: el éxito sigue sin estar garantizado.

3 Punto 3. Temple simulado: aceptación de soluciones peores

Se minimiza $E(x) = x^2$ con $x \in \{-5,\dots,5\}$. Un movimiento con $\Delta E \leq 0$ se acepta siempre. Uno con $\Delta E > 0$ se acepta con probabilidad $e^{-\Delta E/T}$, es decir, cuando $u < P$.

3.1 Parte A: componente matemático y conceptual

1. Los tres casos. Se usa $\Delta E = E(x_{nuevo}) - E(x_{actual})$.

- **Caso A** ($3 \to 2$, $T = 5$, $u = 0{,}70$): $\Delta E = 4-9 = -5$. Como $\Delta E \leq 0$, $P = 1$ y se acepta, porque es una mejora.
- **Caso B** ($2 \to 3$, $T = 5$, $u = 0{,}30$): $\Delta E = 9-4 = 5$ y $P = e^{-5/5} = e^{-1} \approx 0{,}3679$. Como $0{,}30 < 0{,}3679$, se acepta aunque sea peor.
- **Caso C** ($2 \to 3$, $T = 1$, $u = 0{,}30$): $\Delta E = 5$ y $P = e^{-5} \approx 0{,}0067$. Como $0{,}30 > 0{,}0067$, se rechaza.

Caso	**Actual**	**Candidato**	T	u	ΔE	P(**aceptar**)	**Decisión**
A	3	2	5	0,70	−5	1,0000	Acepta
B	2	3	5	0,30	5	0,3679	Acepta
C	2	3	1	0,30	5	0,0067	Rechaza

Tabla 12: Aceptación de cada transición analizada de forma independiente.

2. Influencia de la temperatura. Los casos B y C tienen el mismo ΔE y el mismo u, y solo cambia T. Con T alta, la probabilidad de aceptar un movimiento peor es grande (B). Con T baja es casi nula (C). Una temperatura alta favorece explorar; una baja, conservar lo ya logrado.
3. Enfriamiento demasiado rápido. La temperatura baja antes de que el algoritmo haya explorado. Entonces casi no acepta movimientos peores y se comporta como un ascenso de colinas. Queda atrapado en el óptimo local más cercano a su punto de partida.
4. Comparación con ascenso de colinas. El ascenso de colinas mira a todos los vecinos, elige el mejor y solo se mueve si mejora. El temple simulado evalúa un único vecino aleatorio y puede aceptarlo aunque sea peor, con una probabilidad que depende de T. Esa posibilidad le permite salir de óptimos locales.

3.2 Parte B: implementación en Python

3.2.1 Decisiones de implementación

Aspecto	Supuesto adoptado
Generador	`random.Random(42)`, creado una sola vez al inicio de `temple_simulado`.
Vecinos	Los válidos entre $x-1$ y $x+1$. En los extremos ($x = \pm 5$) hay uno solo y se elige ese, sin consumir llamadas aleatorias.
Orden por iteración	(1) `choice` del vecino, solo si hay dos; (2) $u =$ `random()`, que se extrae siempre, incluso cuando $\Delta E \leq 0$; (3) se acepta si $u < P$; (4) se actualiza la mejor solución solo si mejora estrictamente.
Temperatura	La iteración k usa $T_k = 10 \cdot 0{,}90^{k-1}$; luego se enfría.
Historial	Por iteración: temperatura usada, candidato, ΔE, P, u, decisión, estado y energía actuales, mejor estado y mejor energía. Incluye la fila 0 con el estado inicial ($x_0 = 5$, $E = 25$, que también es la mejor solución hasta ese momento) y 100 filas más de iteraciones. La fila 0 no cuenta como iteración ni decide nada.

Tabla 13: Decisiones de implementación del punto 3.

3.2.2 Código

```python
"""Punto 3: temple simulado para minimizar E(x) = x^2, x en -5..5."""
import json
import math
import random
from pathlib import Path

LIMITE = 5
ESTADO_INICIAL = 5
TEMPERATURA_INICIAL = 10.0
FACTOR_ENFRIAMIENTO = 0.90
ITERACIONES = 100
SEMILLA = 42
CARPETA_SALIDA = Path(__file__).resolve().parent.parent / "outputs"


def energia(x):
```

```
    """Energia E(x) = x^2."""
    return x ** 2


def prob_aceptacion(delta, temperatura):
    """1 si delta <= 0; exp(-delta / T) si delta > 0."""
    if delta <= 0:
        return 1.0
    return math.exp(-delta / temperatura)


def temple_simulado():
    """Ejecuta 100 iteraciones y devuelve el registro completo.

    La fila 0 del historial es el estado inicial; no decide nada.
    """
    generador = random.Random(SEMILLA)
    actual = ESTADO_INICIAL
    mejor = actual
    temperatura = TEMPERATURA_INICIAL
    historial = [{
        "iteracion": 0,
        "temperatura": None,
        "candidato": None,
        "delta": None,
        "probabilidad": None,
        "u": None,
        "aceptado": None,
        "estado_actual": actual,
        "energia_actual": energia(actual),
        "mejor_estado": mejor,
        "mejor_energia": energia(mejor),
    }]
    for iteracion in range(1, ITERACIONES + 1):
        vecinos = [v for v in (actual - 1, actual + 1)
                   if -LIMITE <= v <= LIMITE]
        if len(vecinos) == 1:
            candidato = vecinos[0]
        else:
            candidato = generador.choice(vecinos)
        u = generador.random()
        delta = energia(candidato) - energia(actual)
        probabilidad = prob_aceptacion(delta, temperatura)
        aceptado = u < probabilidad
        if aceptado:
            actual = candidato
        if energia(actual) < energia(mejor):
            mejor = actual
        historial.append({
            "iteracion": iteracion,
            "temperatura": temperatura,
            "candidato": candidato,
            "delta": delta,
            "probabilidad": probabilidad,
            "u": u,
            "aceptado": aceptado,
            "estado_actual": actual,
            "energia_actual": energia(actual),
            "mejor_estado": mejor,
            "mejor_energia": energia(mejor),
        })
        temperatura *= FACTOR_ENFRIAMIENTO
    return {
        "semilla": SEMILLA,
        "estado_inicial": ESTADO_INICIAL,
        "energia_inicial": energia(ESTADO_INICIAL),
        "historial": historial,
        "estado_final": actual,
        "energia_final": energia(actual),
        "mejor_solucion": mejor,
        "energia_minima": energia(mejor),
        "peores_aceptados": sum(
            1 for fila in historial if fila["aceptado"] and fila["delta"] > 0
```

```
        ),
        "final_coincide_con_mejor": actual == mejor,
    }


def main():
    resultados = temple_simulado()
    CARPETA_SALIDA.mkdir(exist_ok=True)
    ruta = CARPETA_SALIDA / "punto3_resultados.json"
    ruta.write_text(json.dumps(resultados, indent=2), encoding="utf-8")
    print("Mejor solucion:", resultados["mejor_solucion"])
    print("Energia minima:", resultados["energia_minima"])
    print("Estado final:", resultados["estado_final"])
    print("Peores aceptados:", resultados["peores_aceptados"])
    print("Estados:", [f["estado_actual"] for f in resultados["historial"]])


if __name__ == "__main__":
    main()
```

Código 3: Implementación del punto 3 (`punto3_temple_simulado.py`).

3.2.3 Resultados

Estado inicial	Mejor solución	Energía mínima	Estado final	Energía final
5	0	0	0	0

Tabla 14: Resultado del temple simulado tras 100 iteraciones.

- **Estructura:** una fila con el estado inicial, la mejor solución, la energía mínima y el estado y la energía finales.
- **Qué se ve:** la mejor solución es $x = 0$ con energía mínima 0. El estado final también es $x = 0$.
- **Por qué:** en esta ejecución el estado final coincide con la mejor solución; el historial muestra que no siempre fue así durante el recorrido.

Iter.	T	Iter.	T	Iter.	T	Iter.	T	Iter.	T
1	10,0000	21	1,2158	41	0,1478	61	0,0180	81	0,0022
2	9,0000	22	1,0942	42	0,1330	62	0,0162	82	0,0020
3	8,1000	23	0,9848	43	0,1197	63	0,0146	83	0,0018
4	7,2900	24	0,8863	44	0,1078	64	0,0131	84	0,0016
5	6,5610	25	0,7977	45	0,0970	65	0,0118	85	0,0014
6	5,9049	26	0,7179	46	0,0873	66	0,0106	86	0,0013
7	5,3144	27	0,6461	47	0,0786	67	0,0096	87	0,0012
8	4,7830	28	0,5815	48	0,0707	68	0,0086	88	0,0010
9	4,3047	29	0,5233	49	0,0636	69	0,0077	89	0,0009
10	3,8742	30	0,4710	50	0,0573	70	0,0070	90	0,0008
11	3,4868	31	0,4239	51	0,0515	71	0,0063	91	0,0008
12	3,1381	32	0,3815	52	0,0464	72	0,0056	92	0,0007
13	2,8243	33	0,3434	53	0,0417	73	0,0051	93	0,0006
14	2,5419	34	0,3090	54	0,0376	74	0,0046	94	0,0006
15	2,2877	35	0,2781	55	0,0338	75	0,0041	95	0,0005
16	2,0589	36	0,2503	56	0,0304	76	0,0037	96	0,0004
17	1,8530	37	0,2253	57	0,0274	77	0,0033	97	0,0004
18	1,6677	38	0,2028	58	0,0247	78	0,0030	98	0,0004
19	1,5009	39	0,1825	59	0,0222	79	0,0027	99	0,0003
20	1,3509	40	0,1642	60	0,0200	80	0,0024	100	0,0003

Tabla 15: Historial de la temperatura usada en cada iteración.

- **Estructura:** cinco bloques de columnas con la iteración y la temperatura usada en ella.
- **Qué se ve:** la temperatura baja de 10,0 en la iteración 1 a $2{,}95 \times 10^{-4}$ en la iteración 100.
- **Por qué:** cada paso multiplica por 0,90, así que la caída es geométrica.

Iter.	x actual	E actual	x mejor	E mejor	Iter.	x actual	E actual	x mejor	E mejor	Iter.	x actual	E actual	x mejor	E mejor
0	5	25	5	25	34	0	0	0	0	68	0	0	0	0
1	4	16	4	16	35	0	0	0	0	69	0	0	0	0
2	3	9	3	9	36	0	0	0	0	70	0	0	0	0
3	2	4	2	4	37	0	0	0	0	71	0	0	0	0
4	1	1	1	1	38	0	0	0	0	72	0	0	0	0
5	0	0	0	0	39	0	0	0	0	73	0	0	0	0
6	−1	1	0	0	40	0	0	0	0	74	0	0	0	0
7	−2	4	0	0	41	0	0	0	0	75	0	0	0	0
8	−2	4	0	0	42	0	0	0	0	76	0	0	0	0
9	−1	1	0	0	43	0	0	0	0	77	0	0	0	0
10	0	0	0	0	44	0	0	0	0	78	0	0	0	0
11	0	0	0	0	45	0	0	0	0	79	0	0	0	0
12	−1	1	0	0	46	0	0	0	0	80	0	0	0	0
13	0	0	0	0	47	0	0	0	0	81	0	0	0	0
14	0	0	0	0	48	0	0	0	0	82	0	0	0	0
15	1	1	0	0	49	0	0	0	0	83	0	0	0	0
16	2	4	0	0	50	0	0	0	0	84	0	0	0	0
17	2	4	0	0	51	0	0	0	0	85	0	0	0	0
18	1	1	0	0	52	0	0	0	0	86	0	0	0	0
19	0	0	0	0	53	0	0	0	0	87	0	0	0	0
20	1	1	0	0	54	0	0	0	0	88	0	0	0	0
21	1	1	0	0	55	0	0	0	0	89	0	0	0	0
22	1	1	0	0	56	0	0	0	0	90	0	0	0	0
23	0	0	0	0	57	0	0	0	0	91	0	0	0	0
24	0	0	0	0	58	0	0	0	0	92	0	0	0	0
25	0	0	0	0	59	0	0	0	0	93	0	0	0	0
26	0	0	0	0	60	0	0	0	0	94	0	0	0	0
27	0	0	0	0	61	0	0	0	0	95	0	0	0	0
28	1	1	0	0	62	0	0	0	0	96	0	0	0	0
29	1	1	0	0	63	0	0	0	0	97	0	0	0	0
30	1	1	0	0	64	0	0	0	0	98	0	0	0	0
31	0	0	0	0	65	0	0	0	0	99	0	0	0	0
32	0	0	0	0	66	0	0	0	0	100	0	0	0	0
33	0	0	0	0	67	0	0	0	0					

Tabla 16: Historial del estado actual y de la mejor solución con sus energías (la iteración 0 es el estado inicial).

- **Estructura:** tres bloques con la iteración, el estado actual y su energía, y la mejor solución y su energía. La iteración 0 es el estado inicial.
- **Qué se ve:** la energía parte de 25 en $x_0 = 5$ (iteración 0), llega a 0 en la iteración 5 y vuelve a subir a 1 en la iteración 6.
- **Por qué:** la mejor energía no sube nunca; la actual sí puede subir cuando se acepta un movimiento peor.

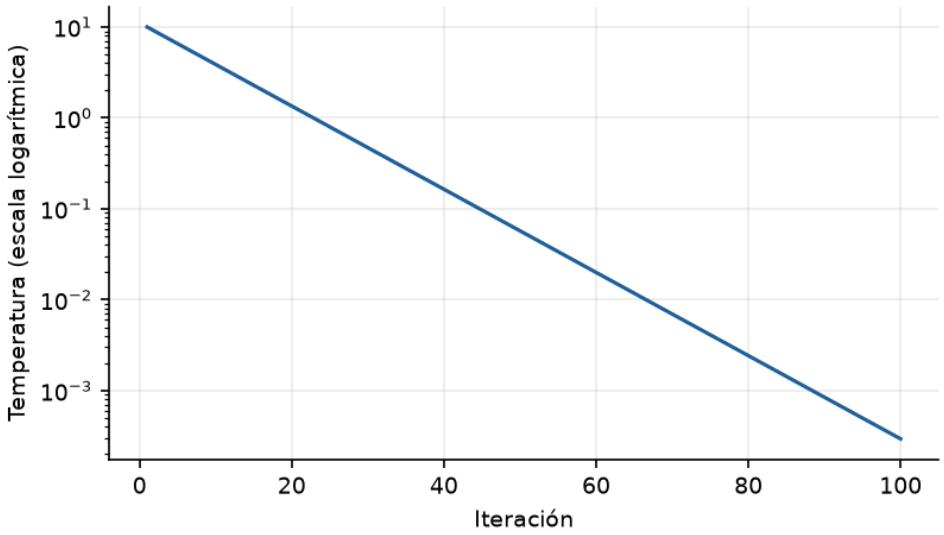


Figura 5: Temperatura por iteración en escala logarítmica.

- **Estructura:** iteraciones en el eje horizontal y temperatura en el vertical, en escala logarítmica.
- **Qué se ve:** una recta descendente.
- **Por qué:** una caída geométrica es una recta en escala logarítmica, con pendiente $\ln 0{,}90$.

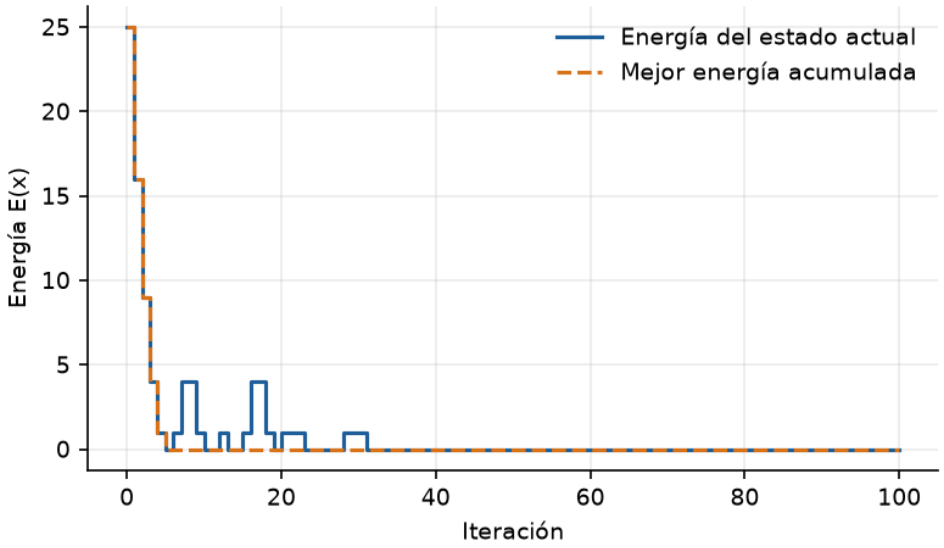


Figura 6: Energía del estado actual y mejor energía acumulada por iteración.

- **Estructura:** dos escalones desde la iteración 0 (estado inicial, $E = 25$): la energía del estado actual (azul) y la mejor energía vista hasta ese momento (naranja punteada).
- **Qué se ve:** el estado actual desciende hasta 0 en la iteración 5, vuelve a subir hasta energía 4 en varias ocasiones y desde la iteración 31 queda fijo en 0. La mejor energía llega a 0 y se queda.
- **Por qué:** al principio la temperatura alta permite subir; al final, con T baja, el estado ya no se mueve de 0.

3.2.4 Pregunta de interpretación

¿Por qué guardar la mejor solución además del estado actual?

- El temple simulado puede aceptar movimientos peores, así que el estado actual puede alejarse de lo mejor que ha visto.
- En esta ejecución el estado final coincide con la mejor solución ($x = 0$), pero eso fue una coincidencia del recorrido, no una garantía del método.
- El historial lo prueba: la mejor energía 0 se alcanzó en la iteración 5, y en la iteración 6 el estado actual ya había subido a energía 1. En 15 de las 100 iteraciones el estado actual fue peor que la mejor solución. Si el algoritmo se hubiera detenido en cualquiera de ellas, devolver solo el estado actual habría entregado una solución peor que una ya encontrada.
- Se aceptaron 7 de 88 candidatos peores, el último en la iteración 28 (con $T = 0{,}58$). En la segunda mitad (iteraciones 51 a 100) hubo 50 propuestas peores y ninguna se aceptó, porque T ya era muy baja. Mientras T es alta, el riesgo de alejarse de lo mejor existe, y guardar la mejor solución lo cubre.

4 Punto 4. ACO: elección de rutas y actualización de feromonas

Una hormiga en el nodo A elige su siguiente nodo con $p_{Aj} = \tau_{Aj}^{\alpha}\eta_{Aj}^{\beta} / \sum_k \tau_{Ak}^{\alpha}\eta_{Ak}^{\beta}$, con $\eta_{Aj} = 1/d_{Aj}$, $\alpha = 1$ y $\beta = 2$.

4.1 Parte A: componente matemático y conceptual

1. Pesos y probabilidades. El peso de cada arista es $\tau\eta^2$. Para $A \to B$ es $1 \cdot (1/2)^2 = 0{,}25$; para $A \to C$, $2 \cdot (1/4)^2 = 0{,}125$; para $A \to D$, $1 \cdot (1/5)^2 = 0{,}04$. La suma de los pesos es 0,415. La probabilidad de cada arista es su peso dividido entre esa suma (Tabla 17).

Arista	d	τ	**Peso** $\tau\eta^2$	**Probabilidad**
$A \to B$	2	1	0,2500	0,6024
$A \to C$	4	2	0,1250	0,3012
$A \to D$	5	1	0,0400	0,0964
Suma			0,4150	1,0000

Tabla 17: Peso y probabilidad de elección desde el nodo A.

2. Alternativa más probable. Es $A \to B$, con 60,24 %. Su elección no está garantizada: la regla es probabilística, así que la hormiga puede ir a C o a D con las probabilidades restantes.

3. Actualización de feromonas. Con $\rho = 0{,}2$, $Q = 10$ y $L = 10$, el depósito en $A \to B$ es $\Delta\tau_{AB} = Q/L = 1$. Las otras dos aristas no reciben depósito. Primero se evapora el 20 % y luego se suma el depósito: $\tau_{AB} = 0{,}8 \cdot 1 + 1 = 1{,}8$, $\tau_{AC} = 0{,}8 \cdot 2 = 1{,}6$ y $\tau_{AD} = 0{,}8 \cdot 1 = 0{,}8$ (Tabla 18).

Arista	τ actual	$(1-\rho)\tau$	$\Delta\tau$	τ nueva
$A \to B$	1,0	0,8	1,0	1,8
$A \to C$	2,0	1,6	0,0	1,6
$A \to D$	1,0	0,8	0,0	0,8

Tabla 18: Actualización de la feromona tras una hormiga con $L = 10$ que usa $A \to B$.

4. Evaporación y dominio de la feromona.

- **Función de la evaporación.** Hace que la feromona de rutas que dejan de usarse se desvanezca. Así el sistema olvida rutas antiguas y evita convergir de forma prematura a una sola.
- **Si la feromona domina.** El término τ^{α} pesa más que la información heurística η^{β}. Entonces las hormigas siguen la ruta con más feromona aunque no sea la mejor. El sistema se estanca y se pierde la exploración de rutas alternativas.

4.2 Parte B: implementación en Python

4.2.1 Decisiones de implementación

Aspecto	Supuesto adoptado
Generador	`numpy.random.default_rng(42)`, creado una sola vez al inicio de `aco`.
Elección de las hormigas	Una sola llamada por iteración: `rng.choice(3, size=10, p=p)`. Las diez hormigas eligen de forma independiente con las mismas probabilidades.
Orden por iteración	(1) probabilidades con las feromonas actuales; (2) elección de las diez hormigas; (3) evaporación de todas las rutas, $\tau \leftarrow 0{,}8\,\tau$; (4) cada hormiga suma Q/L_r a la ruta que eligió.
Registro	Las probabilidades guardadas son las usadas para elegir en esa iteración. Las feromonas guardadas son las del final de la iteración.
Mejor ruta observada	La de menor longitud entre las rutas elegidas al menos una vez.

Tabla 19: Decisiones de implementación del punto 4.

4.2.2 Código

```
"""Punto 4: modelo simplificado de ACO con tres rutas completas."""
import json
from pathlib import Path

import numpy as np

RUTAS = ["A-B-Z", "A-C-Z", "A-D-Z"]
LONGITUDES = np.array([6.0, 8.0, 10.0])
HORMIGAS = 10
ITERACIONES = 20
ALFA = 1.0
BETA = 2.0
RHO = 0.2
```

```
Q = 10.0
SEMILLA = 42
CARPETA_SALIDA = Path(__file__).resolve().parent.parent / "outputs"


def probabilidades(feromonas):
    """p_r = tau^alfa (1/L)^beta normalizado sobre las tres rutas."""
    pesos = feromonas ** ALFA * (1.0 / LONGITUDES) ** BETA
    return pesos / pesos.sum()


def aco():
    """Ejecuta las 20 iteraciones y devuelve el registro completo."""
    generador = np.random.default_rng(SEMILLA)
    feromonas = np.ones(len(RUTAS))
    registro = []
    totales = np.zeros(len(RUTAS), dtype=int)
    for iteracion in range(1, ITERACIONES + 1):
        p = probabilidades(feromonas)
        elegidas = generador.choice(len(RUTAS), size=HORMIGAS, p=p)
        conteo = np.bincount(elegidas, minlength=len(RUTAS))
        feromonas = (1.0 - RHO) * feromonas
        for ruta in elegidas:
            feromonas[ruta] += Q / LONGITUDES[ruta]
        totales += conteo
        registro.append({
            "iteracion": iteracion,
            "probabilidades": p.tolist(),
            "conteo": conteo.tolist(),
            "feromonas": feromonas.tolist(),
        })
    usadas = [i for i in range(len(RUTAS)) if totales[i] > 0]
    mejor_observada = min(usadas, key=lambda i: LONGITUDES[i])
    mas_elegida = int(np.argmax(totales))
    return {
        "semilla": SEMILLA,
        "rutas": RUTAS,
        "longitudes": LONGITUDES.tolist(),
        "probabilidades_iniciales": probabilidades(np.ones(3)).tolist(),
        "registro": registro,
        "frecuencias_totales": totales.tolist(),
        "decisiones": int(totales.sum()),
        "feromonas_finales": feromonas.tolist(),
        "probabilidades_finales": probabilidades(feromonas).tolist(),
        "mejor_ruta_observada": RUTAS[mejor_observada],
        "ruta_mas_elegida": RUTAS[mas_elegida],
    }


def main():
    resultados = aco()
    CARPETA_SALIDA.mkdir(exist_ok=True)
    ruta = CARPETA_SALIDA / "punto4_resultados.json"
    ruta.write_text(json.dumps(resultados, indent=2), encoding="utf-8")
    print("Probabilidades iniciales:", resultados["probabilidades_iniciales"])
    print("Frecuencias totales:", resultados["frecuencias_totales"])
    print("Feromonas finales:", resultados["feromonas_finales"])
    print("Mejor ruta observada:", resultados["mejor_ruta_observada"])
    for fila in resultados["registro"]:
        print(fila["iteracion"], fila["conteo"])


if __name__ == "__main__":
    main()
```

Código 4: Implementación del punto 4 (`punto4_aco.py`).

4.2.3 Resultados

Ruta	Longitud	Frecuencia	Proporción
A-B-Z	6	185	92,5 %
A-C-Z	8	10	5,0 %
A-D-Z	10	5	2,5 %
Total		200	100,0 %

Tabla 20: Frecuencia total de elección de cada ruta en las 200 decisiones.

- **Estructura:** una fila por ruta con su longitud, cuántas veces fue elegida y qué parte de las 200 decisiones representa.
- **Qué se ve:** *A-B-Z* fue elegida 185 veces (92,5 %), *A-C-Z* 10 veces (5,0 %) y *A-D-Z* 5 veces (2,5 %).
- **Por qué:** la ruta más corta concentra casi todas las elecciones.

Ruta	Feromona inicial	Feromona final	Probabilidad final
A-B-Z	1	81,9229	0,9979
A-C-Z	1	0,2356	0,0016
A-D-Z	1	0,1090	0,0005

Tabla 21: Feromona de cada ruta al terminar la iteración 20 y probabilidad de elección que implicaría.

- **Estructura:** una fila por ruta con la feromona inicial, la final y la probabilidad de elección que esa feromona implica.
- **Qué se ve:** la feromona de *A-B-Z* es 81,92, frente a 0,24 y 0,11 en las otras dos.
- **Por qué:** la ruta elegida recibe depósito en cada iteración y las otras solo se evaporan.

Iter.	A-B-Z	A-C-Z	A-D-Z	Iter.	A-B-Z	A-C-Z	A-D-Z
1	4	4	2	11	10	0	0
2	6	3	1	12	10	0	0
3	6	3	1	13	10	0	0
4	9	0	1	14	10	0	0
5	10	0	0	15	10	0	0
6	10	0	0	16	10	0	0
7	10	0	0	17	10	0	0
8	10	0	0	18	10	0	0
9	10	0	0	19	10	0	0
10	10	0	0	20	10	0	0

Tabla 22: Elecciones de cada ruta por iteración (diez hormigas).

- **Estructura:** dos bloques de columnas con la iteración y cuántas de las diez hormigas eligieron cada ruta.
- **Qué se ve:** en la iteración 1 eligen 4, 4 y 2 hormigas; desde la iteración 5 las diez eligen *A-B-Z*.
- **Por qué:** es el momento en que la preferencia pasa de repartida a total.

Iter.	A-B-Z	A-C-Z	A-D-Z	Iter.	A-B-Z	A-C-Z	A-D-Z
1	0,5202	0,2926	0,1873	11	0,9777	0,0172	0,0051
2	0,6362	0,2780	0,0859	12	0,9827	0,0133	0,0039
3	0,7307	0,2159	0,0534	13	0,9865	0,0104	0,0031
4	0,7604	0,1964	0,0432	14	0,9894	0,0082	0,0024
5	0,8448	0,1197	0,0355	15	0,9917	0,0064	0,0019
6	0,8986	0,0783	0,0232	16	0,9934	0,0051	0,0015
7	0,9292	0,0546	0,0162	17	0,9948	0,0040	0,0012
8	0,9486	0,0396	0,0117	18	0,9959	0,0032	0,0009
9	0,9617	0,0295	0,0087	19	0,9967	0,0025	0,0008
10	0,9710	0,0224	0,0066	20	0,9974	0,0020	0,0006

Tabla 23: Probabilidad de elección de cada ruta al inicio de cada iteración.

- **Estructura:** dos bloques con la iteración y la probabilidad de cada ruta al inicio de ella.
- **Qué se ve:** la probabilidad de *A-B-Z* sube de 0,5202 a 0,9974 y las de las otras dos bajan a 0,0020 y 0,0006.
- **Por qué:** la feromona acumulada se suma a la información heurística y cambia las probabilidades iteración a iteración.

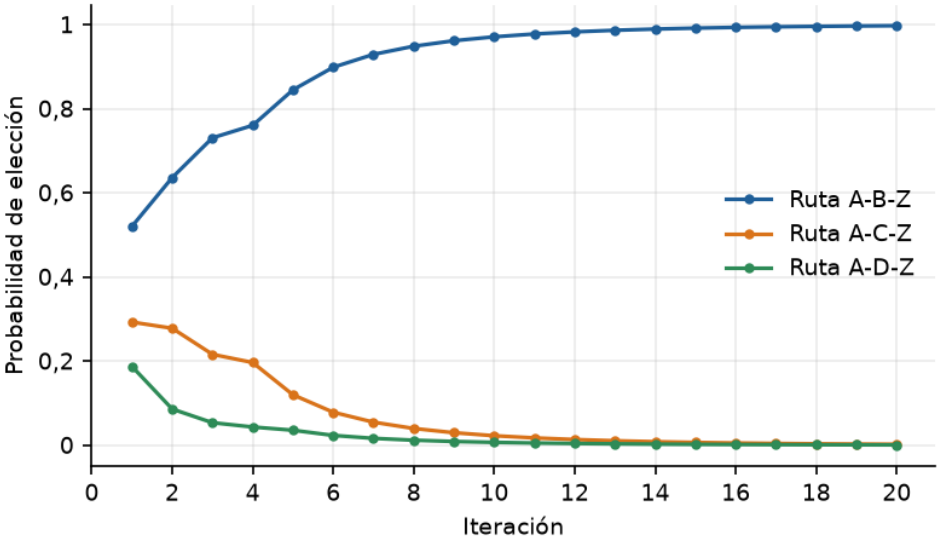


Figura 7: Probabilidad de elección de cada ruta por iteración.

- **Estructura:** una línea por ruta; el eje vertical es la probabilidad con que cada hormiga elige esa ruta en esa iteración.
- **Qué se ve:** la línea de *A-B-Z* sube y se acerca a 1; las otras dos bajan hacia 0.
- **Por qué:** es el efecto de la feromona acumulada sobre la preferencia heurística inicial.

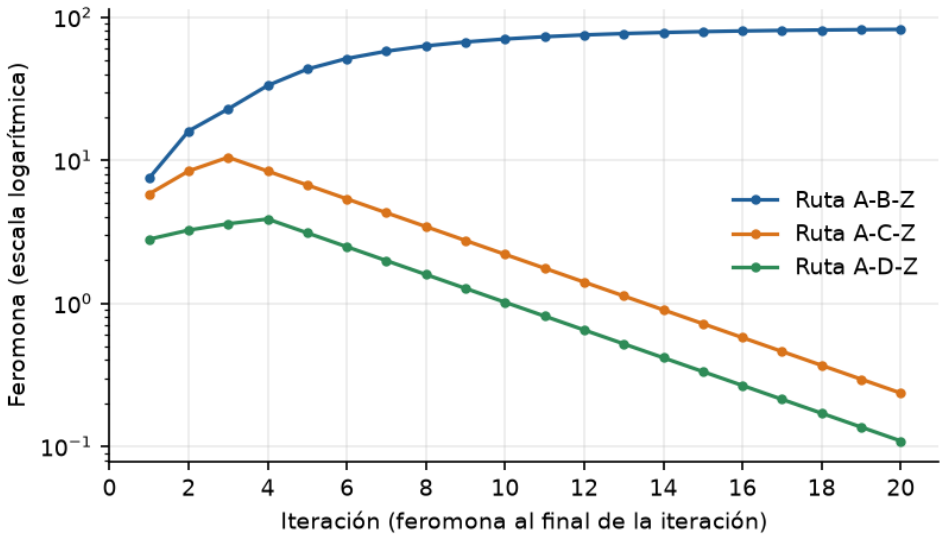


Figura 8: Feromona de cada ruta por iteración en escala logarítmica.

- **Estructura:** una línea por ruta; el eje vertical, logarítmico, es la feromona al final de cada iteración.
- **Qué se ve:** la de *A-B-Z* crece y se estabiliza cerca de 81,92. La de *A-C-Z* sube hasta la iteración 3 (llega a 10,46) y la de *A-D-Z* hasta la 4; después ambas caen en línea recta.
- **Por qué:** sin depósito, la evaporación multiplica por 0,8 en cada iteración, lo que en escala logarítmica es una recta.

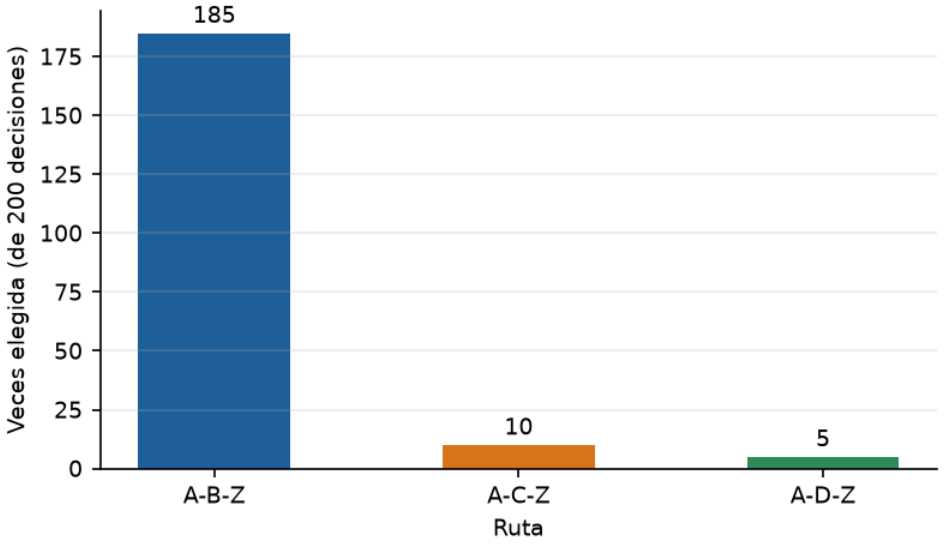


Figura 9: Frecuencia total de elección de cada ruta.

- **Estructura:** una barra por ruta con las veces que fue elegida sobre las 200 decisiones.
- **Qué se ve:** la barra de *A-B-Z* domina y las otras dos son muy bajas.
- **Por qué:** es el resumen de la Tabla 20 en forma gráfica.

4.2.4 Pregunta de interpretación

Mejor ruta observada. Es *A-B-Z*, de longitud 6. Además es la más elegida (185 de 200 decisiones).

Cómo influyen la calidad de las rutas y la experiencia colectiva.

- **Calidad de la ruta, parte heurística.** Con las feromonas iguales en la iteración 1, solo cuenta η^{β}: las probabilidades son 0,5202, 0,2926 y 0,1873. La ruta corta ya parte con ventaja.
- **Calidad de la ruta, parte del depósito.** Cada hormiga deposita Q/L_r: 1,67 en *A-B-Z*, 1,25 en *A-C-Z* y 1,00 en *A-D-Z*. Una ruta corta deja más feromona por cada hormiga que la usa.
- **Experiencia colectiva.** Lo que dejaron las hormigas de una iteración cambia las probabilidades de la siguiente. La de *A-B-Z* pasa de 0,5202 a 0,6362 en la iteración 2, a 0,8448 en la iteración 5 y a 0,9974 en la 20.
- **Refuerzo y olvido.** La ruta preferida recibe más depósito, lo que la hace aún más probable. Las otras solo se evaporan. *A-C-Z* se eligió por última vez en la iteración 3 y *A-D-Z* en la iteración 4.
- **Costo de la convergencia rápida.** Desde la iteración 5 no se explora ninguna otra ruta. Aquí la ruta reforzada es además la más corta, pero el mecanismo no distingue: si una ruta peor hubiera recibido más depósito al inicio, la habría reforzado igual.

5 Punto 5. PSO: movimiento de partículas y memoria

Se minimiza una función con partículas que se mueven con $v_{nuevo} = wv + c_1 r_1 (p_{best} - x) + c_2 r_2 (g_{best} - x)$ y $x_{nuevo} = x + v_{nuevo}$, con $w = 0{,}5$ y $c_1 = c_2 = 1{,}5$.

5.1 Parte A: componente matemático y conceptual

1 y 2. Términos, velocidad, posición y aptitud. La partícula tiene $x = 4$, $v = -1$, $p_{best} = 2$ y $g_{best} = 1$, con $r_1 = 0{,}4$ y $r_2 = 0{,}6$. Los tres términos y el resultado están en la Tabla 24. La partícula pasa de $x = 4$ a $x = -0{,}4$, con $f(-0{,}4) = 0{,}16$.

Concepto	Expresión	Valor
Inercia	$wv = 0{,}5 \cdot (-1)$	$-0{,}50$
Cognitivo	$c_1 r_1 (p_{best} - x) = 1{,}5 \cdot 0{,}4 \cdot (2-4)$	$-1{,}20$
Social	$c_2 r_2 (g_{best} - x) = 1{,}5 \cdot 0{,}6 \cdot (1-4)$	$-2{,}70$
Nueva velocidad	inercia + cognitivo + social	$-4{,}40$
Nueva posición	$x + v_{nuevo} = 4 + (-4{,}4)$	$-0{,}40$
Aptitud	$f(-0{,}4)$	0,16

Tabla 24: Cálculo de un paso de la partícula con $r_1 = 0{,}4$ y $r_2 = 0{,}6$.

3. Actualización de memorias. Se compara la nueva aptitud con la de cada memoria (Tabla 25). Como $0{,}16 < 4 = f(p_{best})$, se actualiza $p_{best} = -0{,}4$. Como $0{,}16 < 1 = f(g_{best})$ y las demás partículas no han mejorado, también se actualiza $g_{best} = -0{,}4$.

Memoria	Aptitud actual	Aptitud nueva	¿Se actualiza?
p_{best}	4,00	0,16	Sí
g_{best}	1,00	0,16	Sí

Tabla 25: Actualización de las memorias personal y global.

4. Mejor posición personal y global. La mejor posición personal es la memoria individual de cada partícula: el mejor punto por el que ella misma ha pasado. La mejor posición global es la mejor de todo el enjambre, y es la misma para todas las partículas.

5. Intercambio de información en PSO y en ACO.

- **En PSO** la información viaja de forma directa: g_{best} es una posición que todas las partículas leen en cada iteración.
- **En ACO** la información es indirecta: las hormigas depositan feromona en el entorno y otras la leen después. Además, esa feromona se evapora, así que la información se desvanece con el tiempo. En PSO, g_{best} se conserva hasta que aparece algo mejor.

5.2 Parte B: implementación en Python

5.2.1 Decisiones de implementación

Aspecto	Supuesto adoptado
Generador	`numpy.random.default_rng(42)`, creado una sola vez al inicio de `pso`.
Orden de llamadas	Primero las posiciones iniciales: `rng.uniform(-5, 5, size=(10, 2))`. En cada iteración, `r1` y luego `r2`, cada una con `rng.uniform(0, 1, size=(10, 2))`. Así r_1 y r_2 son independientes por partícula, dimensión e iteración.
Memorias iniciales	p_{best} igual a la posición inicial de cada partícula. g_{best} igual a la mejor posición inicial.
Actualización	Todas las partículas usan el mismo g_{best} de la iteración. Después se evalúa y se actualizan primero las memorias personales y luego la global, solo con mejora estricta.
Límites	Una coordenada fuera de $[-5,5]$ se ajusta al límite y su componente de velocidad se pone en cero.
Historial	Mejor aptitud global en la iteración 0 (inicial) y tras cada una de las 30 iteraciones: 31 valores.

Tabla 26: Decisiones de implementación del punto 5.

5.2.2 Código

```
"""Punto 5: PSO para minimizar f(x, y) = x^2 + y^2 en [-5, 5]^2."""
import json
from pathlib import Path

import numpy as np

PARTICULAS = 10
DIMENSIONES = 2
LIMITE = 5.0
INERCIA = 0.5
C1 = 1.5
C2 = 1.5
ITERACIONES = 30
SEMILLA = 42
CARPETA_SALIDA = Path(__file__).resolve().parent.parent / "outputs"


def funcion(posiciones):
    """f(x, y) = x^2 + y^2 para cada fila de posiciones."""
```

```
    return np.sum(posiciones ** 2, axis=1)


def pso():
    """Ejecuta las 30 iteraciones y devuelve el registro completo."""
    generador = np.random.default_rng(SEMILLA)
    posiciones = generador.uniform(
        -LIMITE, LIMITE, size=(PARTICULAS, DIMENSIONES))
    velocidades = np.zeros((PARTICULAS, DIMENSIONES))
    posiciones_iniciales = posiciones.copy()
    mejores_personales = posiciones.copy()
    aptitud_personal = funcion(posiciones)
    indice = int(np.argmin(aptitud_personal))
    mejor_global = mejores_personales[indice].copy()
    aptitud_global = float(aptitud_personal[indice])
    historial = [aptitud_global]
    for _ in range(ITERACIONES):
        r1 = generador.uniform(0.0, 1.0, size=(PARTICULAS, DIMENSIONES))
        r2 = generador.uniform(0.0, 1.0, size=(PARTICULAS, DIMENSIONES))
        velocidades = (INERCIA * velocidades
                       + C1 * r1 * (mejores_personales - posiciones)
                       + C2 * r2 * (mejor_global - posiciones))
        posiciones = posiciones + velocidades
        fuera = np.abs(posiciones) > LIMITE
        posiciones = np.clip(posiciones, -LIMITE, LIMITE)
        velocidades[fuera] = 0.0
        aptitudes = funcion(posiciones)
        mejora = aptitudes < aptitud_personal
        mejores_personales[mejora] = posiciones[mejora]
        aptitud_personal[mejora] = aptitudes[mejora]
        indice = int(np.argmin(aptitud_personal))
        if aptitud_personal[indice] < aptitud_global:
            aptitud_global = float(aptitud_personal[indice])
            mejor_global = mejores_personales[indice].copy()
        historial.append(aptitud_global)
    return {
        "semilla": SEMILLA,
        "posiciones_iniciales": posiciones_iniciales.tolist(),
        "posiciones_finales": posiciones.tolist(),
        "velocidades_finales": velocidades.tolist(),
        "historial_mejor_aptitud": historial,
        "mejor_posicion": mejor_global.tolist(),
        "mejor_aptitud": aptitud_global,
        "distancia_al_origen": float(np.linalg.norm(mejor_global)),
        "distancia_media_finales": float(
            np.mean(np.linalg.norm(posiciones, axis=1))),
        "rapidez_media_final": float(
            np.mean(np.linalg.norm(velocidades, axis=1))),
    }


def main():
    resultados = pso()
    CARPETA_SALIDA.mkdir(exist_ok=True)
    ruta = CARPETA_SALIDA / "punto5_resultados.json"
    ruta.write_text(json.dumps(resultados, indent=2), encoding="utf-8")
    print("Mejor posicion:", resultados["mejor_posicion"])
    print("Mejor aptitud:", resultados["mejor_aptitud"])
    print("Distancia al origen:", resultados["distancia_al_origen"])
    print("Historial:", resultados["historial_mejor_aptitud"])


if __name__ == "__main__":
    main()
```

Código 5: Implementación del punto 5 (`punto5_pso.py`).

5.2.3 Resultados

Mejor posición (x,y)	$f(x,y)$	**Distancia al origen**
$(-1{,}848\times10^{-4},\ 1{,}388\times10^{-5})$	$3{,}434\times10^{-8}$	$1{,}853\times10^{-4}$

Tabla 27: Mejor posición global tras 30 iteraciones.

- **Estructura:** una fila con la mejor posición global, su valor de la función y su distancia al origen.
- **Qué se ve:** la mejor posición es $(-1{,}848\times10^{-4},\ 1{,}388\times10^{-5})$, con $f = 3{,}434\times10^{-8}$ y distancia $1{,}853\times10^{-4}$ al origen.
- **Por qué:** está muy cerca de $(0,0)$ pero no es exactamente $(0,0)$.

Iter.	**Mejor aptitud global**	**Iter.**	**Mejor aptitud global**	**Iter.**	**Mejor aptitud global**
0	$7{,}760\times10^{0}$	11	$1{,}024\times10^{-2}$	22	$8{,}881\times10^{-6}$
1	$6{,}533\times10^{-1}$	12	$1{,}819\times10^{-3}$	23	$5{,}834\times10^{-6}$
2	$5{,}698\times10^{-1}$	13	$1{,}058\times10^{-3}$	24	$4{,}319\times10^{-6}$
3	$5{,}698\times10^{-1}$	14	$3{,}262\times10^{-4}$	25	$5{,}965\times10^{-7}$
4	$5{,}216\times10^{-1}$	15	$3{,}262\times10^{-4}$	26	$5{,}965\times10^{-7}$
5	$6{,}750\times10^{-2}$	16	$3{,}262\times10^{-4}$	27	$3{,}434\times10^{-8}$
6	$5{,}232\times10^{-2}$	17	$1{,}008\times10^{-4}$	28	$3{,}434\times10^{-8}$
7	$5{,}232\times10^{-2}$	18	$2{,}402\times10^{-5}$	29	$3{,}434\times10^{-8}$
8	$2{,}148\times10^{-2}$	19	$2{,}402\times10^{-5}$	30	$3{,}434\times10^{-8}$
9	$1{,}268\times10^{-2}$	20	$8{,}881\times10^{-6}$		
10	$1{,}117\times10^{-2}$	21	$8{,}881\times10^{-6}$		

Tabla 28: Historial de la mejor aptitud global (la iteración 0 es la población inicial).

- **Estructura:** tres bloques de columnas con la iteración y la mejor aptitud global.
- **Qué se ve:** la aptitud baja de 7,760 (iteración 0) a 0,653 en la iteración 1 y a $3{,}434\times10^{-8}$ en la iteración 30. Mejoró en 19 de las 30 iteraciones.
- **Por qué:** nunca sube, porque la memoria global solo cambia con mejora estricta.

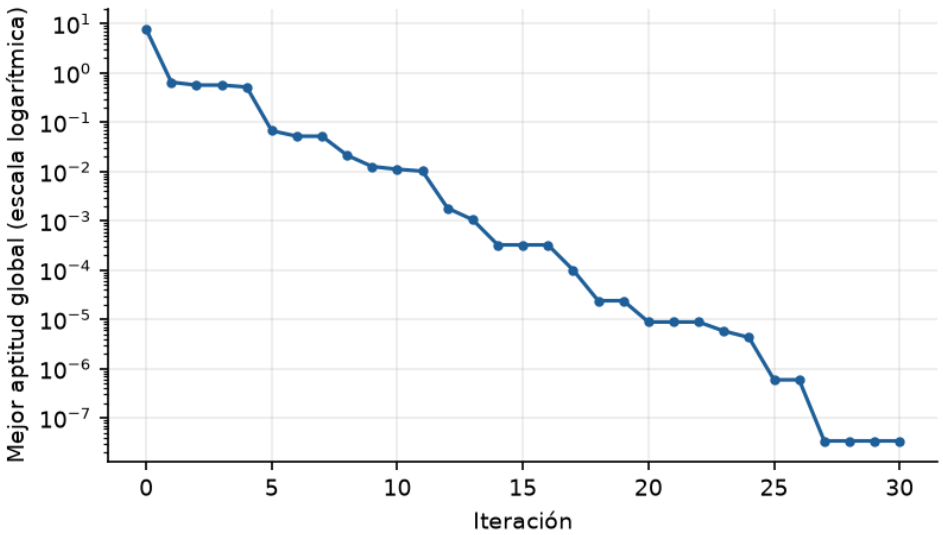


Figura 10: Mejor aptitud global por iteración en escala logarítmica.

- **Estructura:** iteraciones en el eje horizontal y mejor aptitud global en el vertical, en escala logarítmica.
- **Qué se ve:** una curva descendente con tramos planos.
- **Por qué:** los tramos planos son iteraciones sin mejora de g_{best}; la escala logarítmica deja ver que la mejora sigue durante varios órdenes de magnitud.

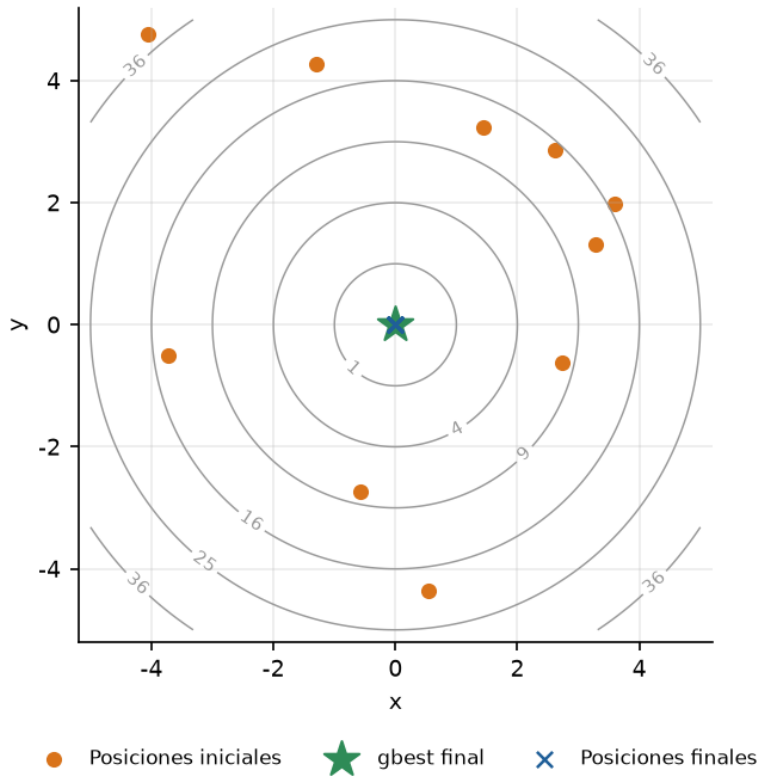


Figura 11: Curvas de nivel de $f(x,y)$ con las posiciones iniciales y finales de las diez partículas y el g_{best} final.

- **Estructura:** círculos concéntricos que son las curvas de nivel $f = 1,4,9,16,25,36$; puntos naranjas (posiciones iniciales), cruces azules (finales) y una estrella verde (g_{best}).
- **Qué se ve:** las posiciones iniciales están dispersas por todo el dominio; las diez cruces finales se agrupan sobre la estrella, en el centro.
- **Por qué:** muestra cómo el enjambre se contrae hacia el mínimo; a esta escala no se distinguen entre sí las partículas finales.

5.2.4 Pregunta de interpretación

¿Por qué convergen sin alcanzar exactamente $(0,0)$?

- **Resultado obtenido.** El mejor punto es $(-1{,}848 \times 10^{-4},\ 1{,}388 \times 10^{-5})$, a $1{,}853 \times 10^{-4}$ del origen, con $f = 3{,}434 \times 10^{-8} > 0$.
- **Espacio continuo.** El dominio tiene infinitos puntos. Las partículas solo visitan los que genera la regla de movimiento, y es prácticamente imposible que alguna caiga exactamente en $(0,0)$.
- g_{best} **es una posición muestreada.** Es el mejor punto que alguna partícula visitó, no el óptimo exacto. El término social atrae a todas hacia ese punto, no hacia $(0,0)$.
- **La velocidad decae pero no se anula.** Con $w = 0{,}5$ la inercia se reduce a la mitad en cada paso, pero los términos cognitivo y social, multiplicados por números alea-

torios, siguen aportando. Al terminar, la rapidez media es $8{,}701 \times 10^{-3}$, distinta de cero.

- **Iteraciones finitas.** Con 30 iteraciones el proceso se detiene antes de llegar más cerca. La última mejora ocurrió en la iteración 27 y la mejora continuaba cerca del final.
- **Componente aleatoria.** Los valores de r_1 y r_2 cambian en cada paso, de modo que las partículas no se posan en un punto: la distancia media de las diez al origen es $3{,}567 \times 10^{-3}$, mayor que la de g_{best}.

6 Conclusiones

- **Algoritmo genético.** Con población de cuatro y elitismo, la mejor aptitud nunca bajó. El óptimo 961 apareció en la generación 16, después de una larga meseta, y solo gracias a una mutación: la población había perdido el único cuarto bit en 1.
- **Ascenso de colinas.** Es determinista y su resultado depende del estado inicial. Llegó al máximo global desde 5 de 9 inicios. Con reinicios, la probabilidad de éxito sube a 0,9122 con tres y a 0,9827 con cinco, sin llegar a 1.
- **Temple simulado.** Aceptó 7 movimientos peores, todos en las primeras 28 iteraciones, y terminó en la mejor solución $x = 0$. Guardar la mejor solución se justifica porque el estado actual estuvo peor que ella en 15 iteraciones.
- **ACO.** La combinación de heurística y feromona llevó a las diez hormigas a *A-B-Z* desde la iteración 5. La convergencia fue rápida y dejó de explorar las otras rutas.
- **PSO.** El enjambre redujo la mejor aptitud de 7,760 a $3{,}434 \times 10^{-8}$, a $1{,}853 \times 10^{-4}$ del origen, sin alcanzarlo exactamente.
- **Exploración y explotación.** En los cinco métodos el equilibrio cambió con el tiempo. La mutación, la temperatura alta, la evaporación y la componente aleatoria sostienen la exploración; la selección, el enfriamiento, la feromona y g_{best} concentran la búsqueda. Los resultados dependen de una sola semilla (42) y no deben generalizarse a todas las ejecuciones.